



Uporedna analiza *database pattern*-a u mikroservisnoj arhitekturi

Pešić Teodora IN1/2022

Todorović Marija IN13/2022

Lukić Nenad IN17/2022

Sančanin Srđan IN59/2022



Od monolita do mikroservisa

Monolitna arhitektura:

- Samostalna celina
- Prednosti : jednostavna implementacija, pronalazak greški, testiranje i isporuka
- Mane : nepraktično za velike sisteme

Mikroservisna arhitektura:

- Skup više manjih, nezavisnih servisa
- Prednosti: skalabilnost, tehnološka heterogenost i lakše održavanje
- Mane: upravljanje i očuvanje konzistentnosti podataka



Pregled *database pattern*-a

- Skup arhitektonskih rešenja za organizaciju, upravljanje i pristup podacima
- Posledica : unapređena skalabilnost sistema, bolje performanse, jednostavnije održavanje i odgovarajući nivo konzistentnosti podataka

01

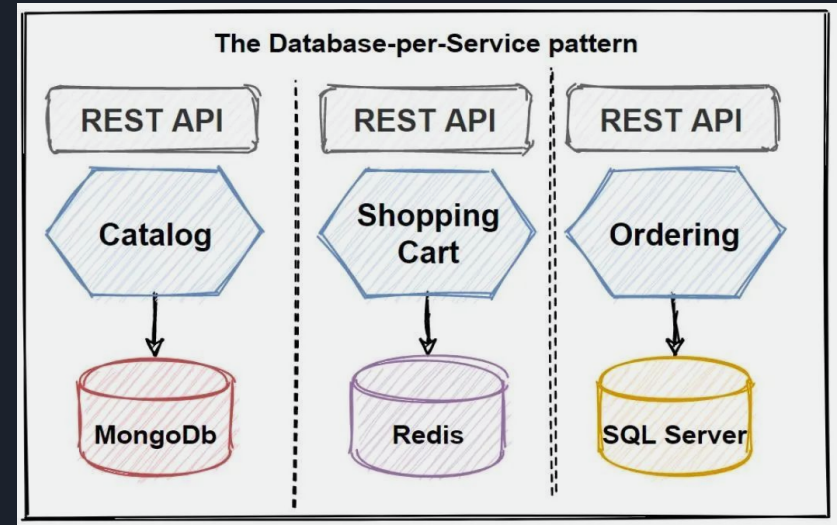
Database per Service

02

Shared Database

Šta je *Database per Service*?

- Svaki servis ima svoju bazu podataka
- Servis je apsolutni vlasnik podataka
- Nezavisnost od drugih servisa i njihovih baza podataka



Slika br. 1: Ilustrovani prikaz sistema koji koristi Database per Service pattern



Database per Service

Prednosti

- Visok stepen autonomije
- Skalabilnost
- *Polyglot Persistence*
- Otpornost sistema na greške

Mane

- Očuvanje konzistentnosti podataka (SAGA i *event-driven* komunikacija)
- Složenost upita koji zahtevaju podatke iz više servisa
- Sinhronizacija promena



Kada koristiti *Database per Service pattern*?

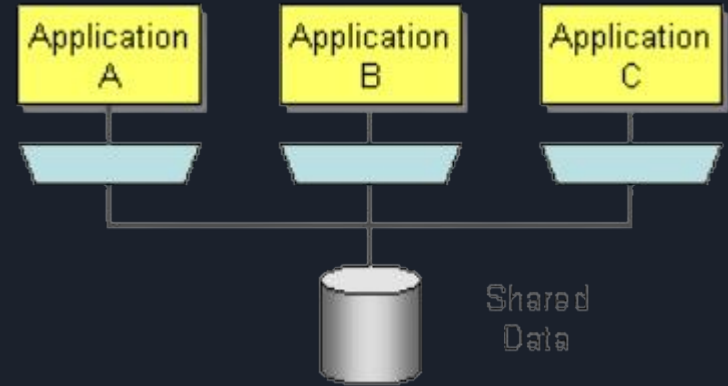
- U velikim distribuiranim sistemima
- Kada je skalabilnost, nezavisnost razvoja i jednostavno održavanje bitno
- Kada je broj korisnika velik
- Kada se zahteva visok nivo dostupnosti

Primeri korišćenja:

- Sistemi za elektronsku trgovinu
- Bankarske aplikacije
- Platforme za rezervacije

Shared Database

- Više nezavisnih mikroservisa deli jednu fizičku bazu podataka ili zajedničku šemu
- Vodi poreklo iz monolitnih sistema
- Ne postoji izolacija šeme
- Čvrsta sprega između servisa (*Coupling*)
- *ACID*



Slika br. 2: Ilustrovani prikaz sistema koji koristi deljenu bazu podataka



Prednosti

- Potpuni inkrementalni backup bez zastoja sistema
- Konzistentni i striktno definisani odnosi između podataka
- Nema potrebe za dodatnim mrežnim interfejsima između servisa
- Manje dupliranja podataka i zastarelih stanja
- Podrška za kompleksne i veoma brze upite
- Transakcije su jednostavnije za implementaciju

Mane

- Baza postaje SPOF
- Veći rizik pri bezbednosnim propustima
- Promena šeme (migracija) se mora raditi vrlo pažljivo
- Potrebna koordinacija puštanja novih verzija između timova
- Teško horizontalno skaliranje za sisteme koji intenzivno upisuju u bazu
- Jedan „težak“ upit može usporiti ceo sistem



Kada koristiti *Shared Database (anti)pattern*?

Kad je tim manji, rani razvoj proizvoda, brzina isporuke je važnija od autonomije servisa

Postojeći sistem je već zasnovan na jedinstvenom relacijskom modelu

Potrebna je stroga *ACID* konzistentnost preko entiteta i ne može se koristiti eventualna konzistentnost

Česti izveštaji na zahtev i kompleksni *JOIN* upiti nad aktivnim podacima su ključna funkcionalnost proizvoda

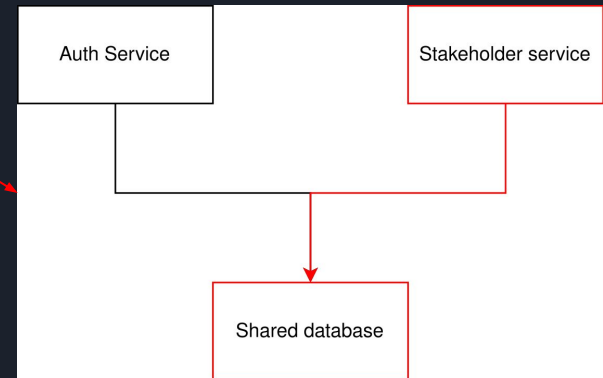
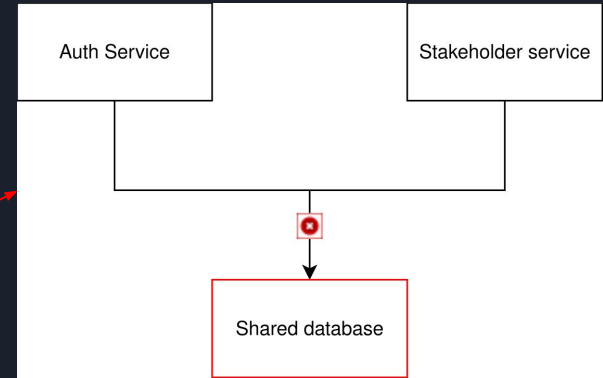
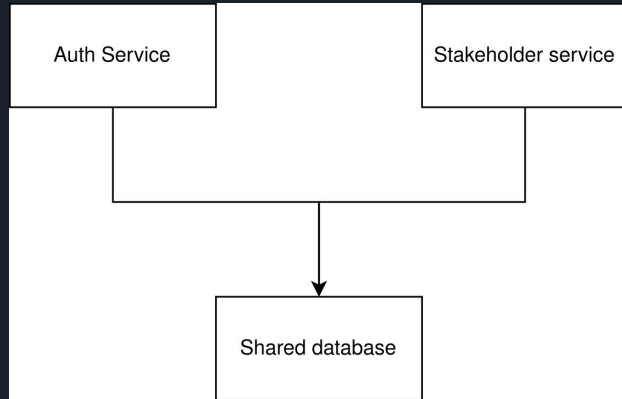
Migracija iz monolita gde je potreban bezbedan prvi korak pre razdvajanja baza



Database per Service ili Shared Database

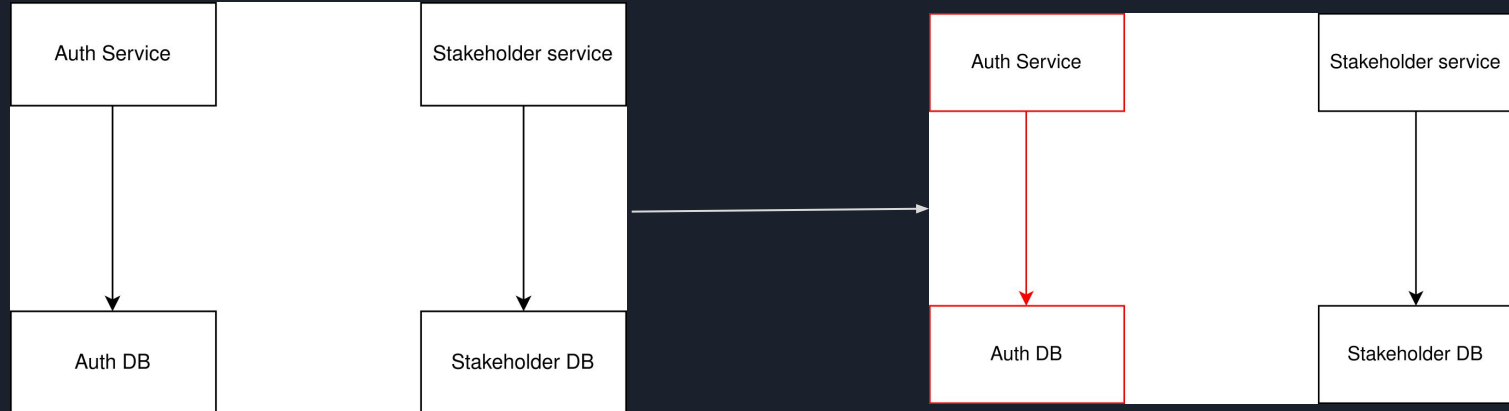
	<i>Database per Service</i>	<i>Shared Database</i>
Autonomija servisa	Visoka	Niska
Konzistentnost podataka	Eventualna konzistentnost	<i>ACID</i> - lokalne transakcije
Skalabilnost	Visoka	Niska
Kompleksnost	Visoka	Niska (u početku)

Shared Database POF



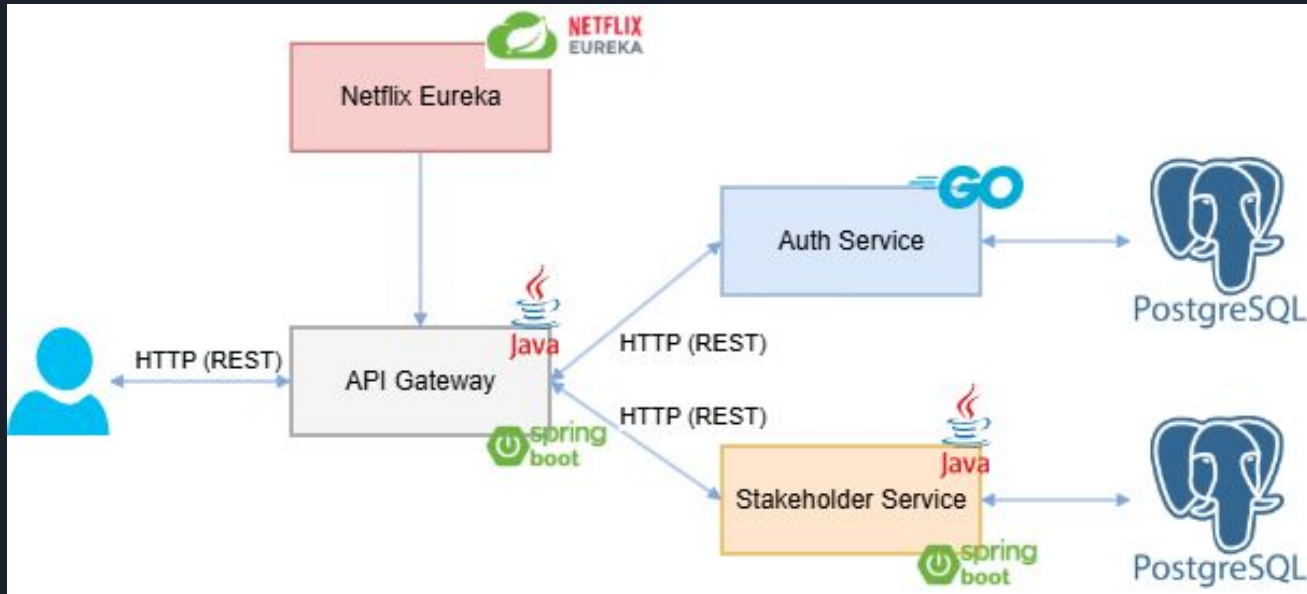
Dijagram br. 1: Ilustrovani prikaz Shared Database POF

Database per Service POF



Dijagram br. 2: Ilustrovani prikaz *Database per Service POF*

Primena *Database per Service* pristupa



Slika br. 3: Prikaz mikroservisne arhitekture za *Database per Service* pristup



Specifičnosti šema baza podataka u *Database per Service* pristupu

- Jednostavnije šeme
- “Ograničenje stranog ključa”
- Samo neophodna polja za poslovne funkcionalnosti
- Očuvanje konzistentnosti podataka

users (Stakeholder Service)	
PK	<u>id : BIGSERIAL</u>
UQ	registry_id : BIGINT NOT NULL
UQ	username : VARCHAR(32) NOT NULL
UQ	email : VARCHAR(255) NOT NULL
role : VARCHAR(32) NOT NULL first_name : VARCHAR(255) last_name : VARCHAR(255) profile_image : VARCHAR(255) biography : TEXT motto : VARCHAR(255) is_blocked : BOOLEAN NOT NULL DEFAULT false	

users (Auth Service)	
PK	<u>id : BIGSERIAL</u>
UQ	username : VARCHAR(32) NOT NULL
UQ	email : VARCHAR(255) NOT NULL
role : VARCHAR(32) NOT NULL password : TEXT NOT NULL	

Slika br. 4: Prikaz šema baza podataka u servisima



Upotreba transakcija u mikroservisnoj arhitekturi

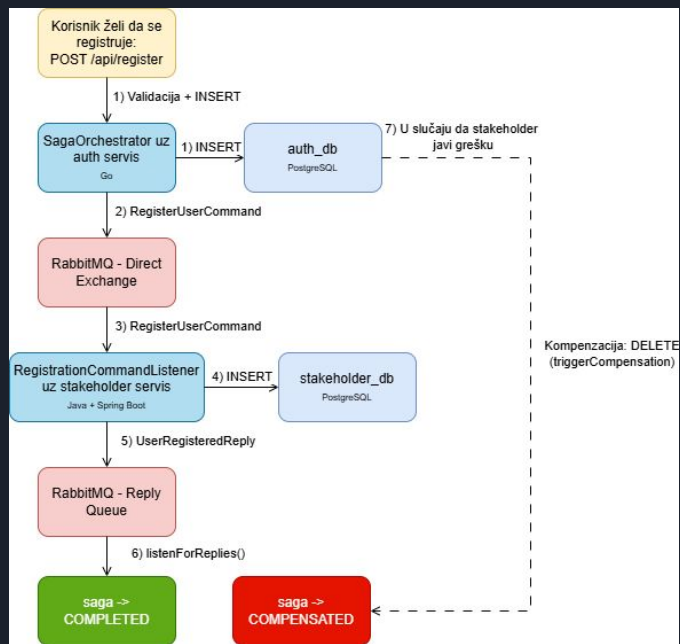
2PC protokol

- Distribuirani algoritam za upravljanje procesima
- Postojanje 2 faze:
 - Faza glasanja
 - *Commit* faza
- Osobine:
 - *ACID* principi
 - Sinhrono izvršavanje
 - Povećanje spregnutosti između servisa
 - Zavisnost od najsporijeg servisa

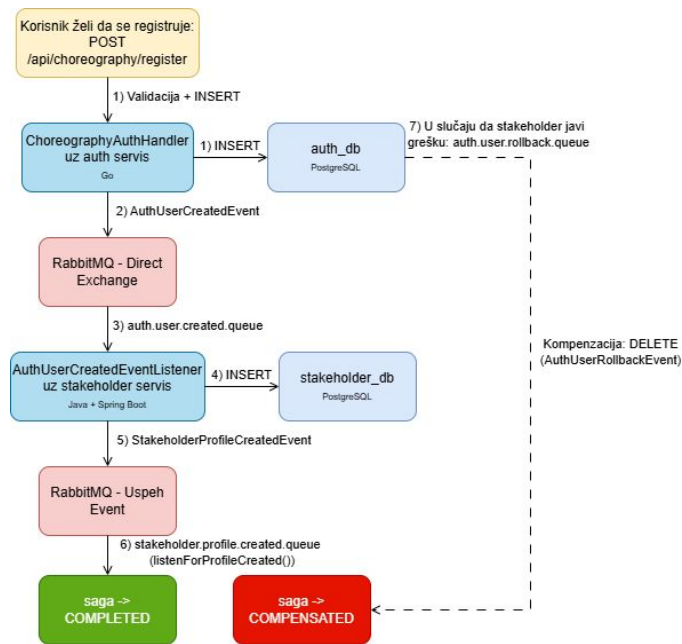
SAGA šablon

- Razlaganje transakcije na niz manjih, lokalnih transakcija
- Primena kompenzacionih akcija u slučaju neuspješnog izvršavanja
- Načini implementacije:
 - Orkestracija
 - Koreografija
- Osobine:
 - *BASE* principi
 - Asinhrono izvršavanje

Primeri implementacije SAGA šablona



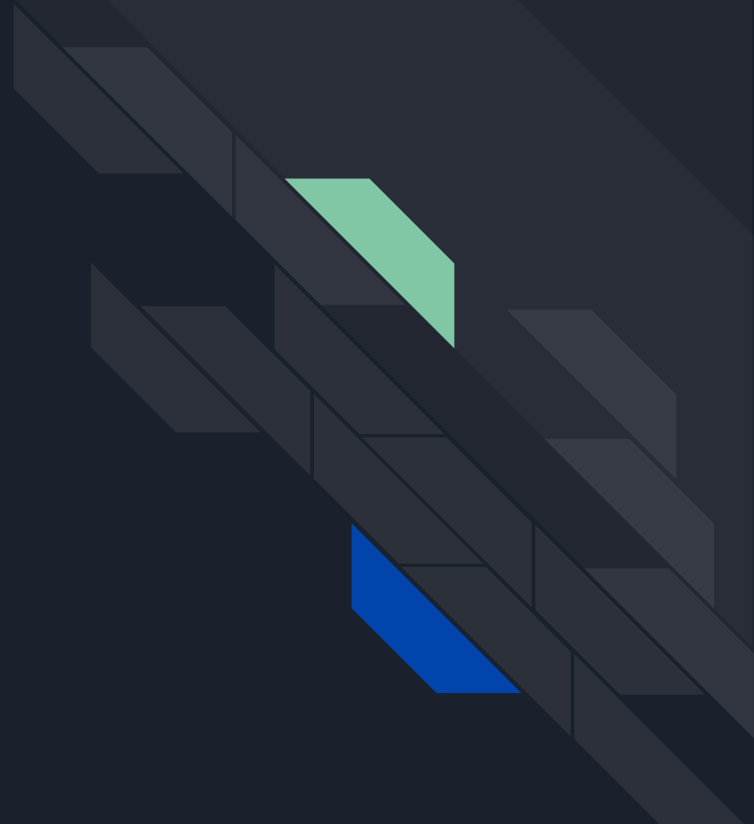
Slika br. 5: Prikaz implementacije orkestriranog SAGA šablona



Slika br. 6: Prikaz implementacije koreografisanog SAGA šablona

O osobinama *Database per Service* pristupa

- Glavni ciljevi pristupa:
 - Nezavisnost baza podataka
 - Enkapsulacija
 - Izbor tehnologija
 - Skalabilnost
 - Otpornost na pad baze podataka
- Najvažnije osobine:
 - Niska spregnutost servisa
 - Nezavisno skaliranje baza podataka i samih servisa, *API Gateway* dobija ulogu *load balancer-a*
 - Autonomija servisa
 - Izolacija greške
 - Dostupnost
 - Kompleksnost i cena razvoja





Hvala na pažnji!

Pitanja?